AeroGuard: Federated Intrusion Detection for ADS-B Surveillance with a Synthetic-Attack Leakage Audit and Class-Prior Correction

Author Name(s)
Affiliation(s)
Email(s)

***Abstract*—Automatic Dependent Surveillance–Broadcast (ADS-B) messages carry no authentication, and the surveillance feeds that could train a learned detector are held by separate receiver operators. We study federated intrusion detection on real OpenSky Network state vectors from four Mondays of 2019, one per quarter, into which five types of synthetic attack are injected: GPS spoofing, velocity spoofing, replay, ghost aircraft and message flooding. Because synthetic attacks can leave traces of their own construction in engineered features, we first audit every feature for label dependence, non-causal computation and construction artifacts, correct the pipeline, and define a core feature set without the identified artifacts. A one-dimensional convolutional neural network (CNN) is trained with FedAvg across – geographic region-cell clients and compared with the same CNN trained centrally on identical rows and with centralized LightGBM. Models are selected on training-day validation data only and tested on four December Mondays over – seeds. With a Bayesian class-prior correction estimated from the training days, the federated CNN has a false alarm rate of –% and an average attack recall of –%, against –% and –% for the centralized CNN. [PENDING RESULTS: one sentence on the core-set ablation and on which attack classes remain difficult.] All attacks are synthetic, so the results characterize detection of the specified injection rules, not of real adversaries.**

***Index Terms*—ADS-B, federated learning, intrusion detection, synthetic attacks, data leakage, class-prior correction, OpenSky Network.**

I. Introduction

Air traffic management depends on accurate surveillance. ADS-B supports it by having each aircraft broadcast its identity, position, velocity and altitude to ground stations and other aircraft [1]; in the United States, ADS-B Out equipment has been required in designated airspace since 1 January 2020 [2]. The messages are neither encrypted nor authenticated, so a receiver cannot establish their origin or integrity from the message alone, and inexpensive radios suffice to inject false positions, fabricated aircraft, replayed messages or bursts of duplicates [3]–[6]. Proposed countermeasures, including authenticated broadcast schemes, remain research proposals that would require changes to avionics and ground infrastructure [5], [7]. A behavioral detector, which asks whether an observation is consistent with an aircraft's motion and with surrounding traffic, could be added at receivers without protocol changes. It complements protocol security; it does not replace it or any certified safety system.

Training such a detector meets two obstacles. First, surveillance feeds are collected by many independent operators. Public crowdsourced feeds such as the OpenSky Network [8] are pooled centrally, but operators' own feeds may not be shareable for commercial or contractual reasons; federated learning keeps raw observations at each participant and exchanges only model parameters [9]. Here the public OpenSky data serve as a stand-in for such separately held feeds. Second, labeled ADS-B attack traffic is essentially unavailable, so studies inject synthetic or simulated attacks into real benign traffic [10]–[12]. Synthetic labels are controllable, but engineered features can then encode how the attack was generated rather than how it would appear to a receiver, and a model can score highly by detecting the generator. Such leakage is a recognized source of overoptimistic results in machine learning for security [13], [14].

This paper treats that risk as part of the evaluation. We train on real OpenSky state vectors with five injected attack types and ask: (i) how much measured performance depends on features that encode the attack construction, and on an earlier, uncorrected version of the pipeline; (ii) whether performance carries over to later days in a different season, and to aircraft not seen in training; (iii) what federated training costs relative to centralized training of the same architecture; and (iv) which attack types remain hard to detect, including under realistic attack prevalence.

The contributions are as follows.

1) A feature-level leakage audit of a synthetic ADS-B attack pipeline, covering label dependence, use of future and test-day observations, and construction artifacts, with the resulting corrections, rule baselines that quantify generator-driven detection, and a comparison of the corrected and uncorrected pipelines (Section IV).
2) A geographically partitioned FedAvg evaluation over naturally non-IID region-cell clients, with a centralized CNN of identical architecture trained on the same rows, so that the federated-versus-centralized gap is measured without a model-family confound, and centralized LightGBM as a separate reference (Sections V and VII).
3) An evaluation protocol that selects models on training-day data only, tests on four later days, reports variability over – seeds with paired comparisons, and analyzes a Bayesian class-prior correction over a predefined grid

of strengths, including when the deployment prevalence differs from the training mix (Section VI).

II. Related Work

A. Learned Detectors for ADS-B

ADS-B threats include eavesdropping, jamming, message injection (ghost targets and flooding), deletion and modification [4]–[6]. Learned detectors fall into three groups. Sequence models learn normal trajectories and flag large prediction or reconstruction errors: an LSTM encoder-decoder over recorded flights [15], an LSTM next-step predictor with a residual threshold [11], and a non-recurrent autoencoder [16]. Supervised classifiers on message features include support vector machines on OpenSky traffic [17], recurrent classifiers [18], tree ensembles [19], deep tabular models trained on OpenSky benign traffic with simulator-generated attacks [12], and sequence encoders pretrained on benign OpenSky state vectors and fine-tuned on injected attacks [20]. A separate line uses physical-layer signal features, which detect replayed and injected transmissions recorded over the air [21]. Sequential collaborative detection across ground and air nodes has also been proposed [22]; it combines detection at several nodes but does not train a shared model.

Table I summarizes the closest studies. Apart from physical-layer work under laboratory conditions [21], every study we found trains and evaluates on synthetic or simulated attack labels, as this one does, and none of them reports an audit of whether engineered features encode the attack construction, although several obtain near-perfect scores.

B. Federated Learning and Federated Intrusion Detection

FedAvg broadcasts a global model, lets clients train copies locally and replaces the global parameters by a sample-weighted average [9]. Horizontal federation applies when clients share a feature schema but hold different records [24]. Non-IID client data can degrade FedAvg [25], [26]. Keeping records local is not a formal privacy guarantee: shared updates can leak training data [27], which secure aggregation [28] and differential privacy [29] are designed to limit. Federated intrusion detection has been studied for IoT device behavior [30], industrial IoT [31], intelligent transportation [32] and smart grids [33], with a recent survey in [34]; results in these settings depend on the data partition, the local schedule and the evaluation split.

Federated learning has received little attention for ADS-B. The only federated ADS-B detector we found, FL-Guard [23], is a preliminary poster study that trains a small network with FedAvg across four simulated aircraft on 4,000 messages from one airport, evaluates alteration and spoofing, and reports no centralized baseline. We found no study that trains a federated detector over geographically partitioned receiver data at the scale of the OpenSky Network, compares it with a centralized model of identical architecture, or audits synthetic-attack features for leakage. Our search covered web and repository sources rather than a formal database review, so this statement describes the literature we found rather than a claim of priority.

C. Attack Construction, Leakage and Prior Correction

Li *et al.* propose a threat model and construction strategy for ADS-B attack data covering tampering, ghost injection, replay and data-block (denial-of-service) patterns injected into real traffic [10]. Wang *et al.* inject random-noise and fixed-offset position perturbations, route replacement, altitude and speed offsets, heading changes and climb-rate changes [11]. Our generation rules were designed independently, and their parameters are not intended to match those studies. More generally, oversampling such as SMOTE [35] and generative models [36] synthesize minority-class samples, and none of these approaches establishes that the generated records resemble adaptive adversaries. Leakage, meaning information available during dataset construction but not at prediction time, inflates measured performance [14], and in security datasets it includes artifacts of how data were collected or labeled [13]. Finally, a classifier trained on a class mix that differs from deployment can be corrected under the label-shift assumption by reweighting its posteriors with the ratio of deployment to training priors [37], [38].

III. Data and Attack Injection

A. Source Data and Splits

The source data are historical state vectors from the OpenSky Network [8], which publishes one Monday per week as hourly files of per-aircraft snapshots, a format also used by recent ADS-B detection studies [20]. Of the 52 Mondays of 2019, the last four are held out for testing. From the remaining complete Mondays, every twelfth is used for training, which gives one day per quarter (Table II). Every test day falls after every training day, and test days are used for nothing but the reported scores. The split is not aircraft-disjoint, since the same airframes fly throughout the year; Section VII-D reports results restricted to test aircraft that never appear in the training days.

Loading all training rows is not tractable on the hardware used, so the loader streams the training files and keeps a uniform random sample of at most a fixed number of rows per (region cell, class) bucket, sized from a global budget of 10^7 rows. Buckets below the cap are kept whole. Table III reports the class mix of the natural training days, the training sample and the test sample, the share of the training sample drawn from each day, and the number of distinct aircraft in each. The cap shifts the class mix of the training sample away from natural traffic, which together with the class weights of Section V motivates the prior correction of Section VI-C. The test sample is not capped.

B. Region Cells

Each record is assigned to a cell of a $10^\circ \times 10^\circ$ latitude and longitude grid, and each cell is one federated client standing in for a regional receiver operator. Cells with fewer than 3,000 sampled rows are dropped, and their rows are excluded from all three models so that every model trains and validates on the same records (Section V).

TABLE I
CLOSEST RELATED STUDIES (N/R: NOT REPORTED)

Study	Year	Benign data	Attack labels	Attacks	Model	Fed.
Habler and Shabtai [15]	2018	FlightRadar24 recordings	Synthetic	Noise, route replacement, altitude drift	LSTM encoder-decoder	N
Ying *et al.* [21]	2019	Own receivers	Real RF, lab	Replay, ghost injection	Two-stage DNN, physical layer	N
Li and Wang [22]	2019	Real ADS-B	n/r	Deviation injection, DoS	Sequential collaborative detection	N
Wang *et al.* [11]	2020	Decoded real ADS-B	Synthetic	10 types incl. offsets, route, heading, climb rate	LSTM predictor	N
Fried and Last [16]	2021	Public flight recordings	Synthetic	Trajectory modification	Autoencoder	N
Ould Slimane *et al.* [17]	2022	OpenSky	Synthetic	Path modification, ghost, velocity drift	SVM	N
Khan *et al.* [23]	2022	One airport, 4,000 messages	Simulator	Alteration, spoofing	MLP	Y
Talaei Khoei *et al.* [18]	2024	Real ADS-B	Synthetic	Path modification, ghost, velocity drift	GRU, LSTM	N
Ahmed *et al.* [12]	2025	OpenSky	Simulator	9 types incl. ghost, delay, spoofing	DeepGBM, TabNet	N
Ngamboé *et al.* [20]	2025	OpenSky	Synthetic	Gradual altitude, speed, heading drift	xLSTM, transformer	N
This work		OpenSky, 4 quarters	Synthetic	GPS, velocity, replay, ghost, flooding	CNN, LightGBM	Y

TABLE II
DATASET PARTITION

Role	Dates (2019)	Rows used
Training	7 Jan, 1 Apr, 24 Jun, 30 Sep	–
Validation	20% of each client's sampled rows	–
Test	9, 16, 23, 30 Dec	–

Training and validation rows are a capped stratified sample of – raw training-day rows. Test rows are a uniform 1% sample of the four test days (seed 42), preserving the natural class mix.

TABLE III
CLASS MIX AND SPLIT SIZES

	Training days	Training sample	Test
Benign	–	–	–
Flooding	–	–	–
Ghost aircraft	–	–	–
GPS spoof	–	–	–
Replay	–	–	–
Velocity spoof	–	–	–
Rows	–	–	–
Aircraft (icao24)	–	–	–
Sample from –		–	
Sample from –		–	
Sample from –		–	
Sample from –		–	

Class shares in %. Training days: all rows of the four training files (the prior used for correction). Training sample: the capped sample models are trained on (– sampler). Test: the held-out sample. The last rows give the share of the training sample drawn from each training day.

C. Attack Generation

Each processing chunk is one hourly file. After cleaning and feature engineering on clean data, all rows are labeled benign. A random generator, re-seeded with 42 at the start of every chunk, permutes row positions and assigns five disjoint slices, each a fraction 0.02 of the chunk, to `gps_spoof`, `velocity_spoof`, `replay`, `ghost_aircraft` and `flooding` in that order. Because the generator is re-seeded per chunk, the same sequence of random draws is applied to every hourly file. Selection is per row, not per aircraft, so an aircraft's history can mix benign and attack rows; the data represent isolated perturbations rather than coherent attack episodes. Table IV lists all constants.

For `gps_spoof`, independent offsets uD_s and vD_s with $u, v \sim \mathcal{U}(-1, 1)$ are added to latitude and longitude, where D_s is the tier limit; the latitude offset reaches about 5.5, 55 and 220 km for the three tiers, and the longitude offset is smaller by the cosine of the latitude. The spoofed position is assigned to whatever region cell it falls in, so a spoofed row can move to a different client. For `velocity_spoof`, the broadcast velocity is multiplied by $m \sim \mathcal{U}(1.1, 8.0)$ and $b \sim \mathcal{U}(5, 40)$ m/s is added to the vertical rate. For `replay`, time and last-contact time are shifted back by the same integer delay; the last-position-update time is not shifted. For `ghost_aircraft`, the row's ICAO address is replaced by a fabricated one; no other field is modified. For `flooding`, k copies of the selected row are appended at times $t + j\delta$, $j = 1, \ldots, k$. Only the appended copies are labeled `flooding`; the original message is unmodified real traffic and stays `benign` (Section IV-C).

The four attacks that relabel existing rows have equal natural shares. Flooding adds rows and so has a larger share, which makes it dominate any metric pooled over attack rows; the evaluation therefore reports per-class and class-averaged metrics (Section VI).

IV. FEATURES AND LEAKAGE AUDIT

A. Features

Rows are sorted by aircraft and time within each chunk, and per-aircraft features restart at every chunk boundary, so the first message of each aircraft in each hourly file has undefined kinematic features. Let $\Delta t = t - t_{\text{prev}}$. Rate features divide by

$$\Delta t_{\text{denom}} = \begin{cases} \text{NaN}, & \text{first message in chunk,} \\ \max(\Delta t, 0.1\,\text{s}), & \text{otherwise.} \end{cases} \quad (1)$$

Table V defines the thirteen features and gives the audit outcome for each. Missing values are filled with -1 before standardization.

B. Leakage Audit

Each feature was checked against four questions: whether it reads the label or a generation flag; whether it uses observations after the current message; whether it uses statistics computed over the day being evaluated; and whether its value

TABLE IV
ATTACK-GENERATION CONSTANTS

Constant	Value	Meaning
RNG seed	42	Re-seeded at the start of every chunk
Per-type rate	0.02	Fraction of chunk rows selected for each attack type
GPS tiers	0.05, 0.5, 2.0 degrees	Maximum offset per axis and tier; tier probabilities 0.34, 0.33, 0.33
Velocity multiplier	$\mathcal{U}(1.1, 8.0)$	Multiplier on broadcast velocity
Vertical-rate boost	$\mathcal{U}(5.0, 40.0)$ m/s	Added to broadcast vertical rate (missing treated as 0)
Replay shift	integer $[2, 900)$ s	Backward shift applied to time and last-contact time
Flooding duplicates	integer $[2, 11)$	Duplicates appended per selected row (2 to 10)
Duplicate spacing	$\mathcal{U}(0.01, 0.5)$ s	Time step between successive duplicates
Ghost address	`0xf00000` to `0xfffffe`	Uniformly drawn fabricated ICAO address; collisions with real addresses not checked
Chunk size	1 hourly file	Unit of selection and feature computation

is set by how an attack is constructed rather than by how the attack would appear to a receiver. Each feature was also scored empirically by how well it alone separates each attack class from benign traffic: the ROC AUC of the feature and of its missing-value indicator, with features flagged at AUC ≥ 0.95 or indicator AUC ≥ 0.9. The audit was run on one processed training day and on the test sample.

The audit led to four corrections. (a) `track_message_count` previously counted all messages from an address in the chunk, including later ones, which a receiver cannot know at detection time; it now counts only messages seen so far. (b) `squawk_changed` previously compared each code with a missing predecessor on every aircraft's first row, so all first rows and all missing codes counted as changes; it now requires both codes to be present. (c) Benign rows with implied speed above 400 m/s were removed as sensor glitches while attack rows at the same speed were kept, which made `is_physically_implausible` a one-sided attack indicator. No rows are removed any more, since such glitches are real traffic that a deployed detector would also receive, and the flag is the same threshold for every row. (d) The original message of each flooding group is now labeled benign (Section IV-C).

Several construction artifacts cannot be removed by recomputation. The velocity-spoof vertical-rate boost has a floor of 5 m/s, above the 2.5 m/s climb threshold, so `is_climbing_or_descending` is set by construction for every velocity-spoof row. A fabricated ghost address usually appears once, so its kinematic features are undefined and its message count is one. Flooding duplicates arrive 0.01 to 0.5 s apart, whereas OpenSky state vectors are snapshots taken at a fixed interval per aircraft, so such gaps do not occur in benign data; a real flooding attack does add messages faster than normal, but the strength of this signal here reflects the sampling of the dataset. [TO CHECK ON DATA: snapshot interval from the benign inter-arrival distribution of a processed day.] Finally, `region_avg_inter_arrival` is a per-cell average over the whole clean chunk, so it uses later messages and, on test days, the test day itself; and because spoofed positions are re-assigned to cells, `gps_spoof` rows can receive the baseline of a cell that their aircraft does not occupy, or a fallback value where the cell has no baseline. For offsets uniform within $\pm D_s$ on each axis and 10° cells, about 8.1% of `gps_spoof` rows are expected to change cell (and client), averaged over the three tiers; the fallback share depends on coverage and is reported in Table V. The first message of each aircraft in a chunk, which has undefined kinematic features in benign traffic as well, makes up [TO CHECK ON DATA: share of first-in-chunk benign rows on a processed day] of benign rows.

The core feature set (Table V) keeps the five per-message kinematic and timing features, the stale-contact flag and the regional baseline, and excludes the identity-history count, the transponder-code and address-format flags, the coverage flag and the two threshold flags with known artifacts. Apart from the chunk-level regional baseline, its features use only the current and previous message of the same aircraft. Every model is trained on both sets (Section VII-B). The core set cannot remove the ghost and flooding signatures, which arise from the generators themselves. To quantify them, two rules that use no learning are evaluated on the test set: "ghost if all four rate features are missing" and "flooding if the gap to the previous message is below 1 s" (Table XV).

C. The Flooding Artifact

An earlier version of the pipeline replaced time gaps shorter than 1 s with missing values before dividing. For flooding duplicates, which are under 0.5 s apart, all four rate features were then missing, and the -1 fill produced a token $(-1, -1, -1, -1)$ that identified flooding. A flooding group consists of one original message and k duplicates with k uniform on $\{2, \ldots, 10\}$, and the original was then labeled `flooding`, so the expected share of flooding rows carrying the token was approximately

$$\frac{\mathbb{E}[k]}{1 + \mathbb{E}[k]} = \frac{6}{7} \approx 0.857, \tag{2}$$

matching the measured 0.8575 (the ratio of expectations approximates the expected ratio). A diagnostic on six hours of 14 January processed with the earlier pipeline (a training day of the earlier model, and not part of the data used here) showed that the earlier model relied on the token: replacing only the four token values by their training means, with the inter-arrival time unchanged, reduced mean flooding confidence from 0.89 to 0.00 on 300 rows, and among all 1,143,664 flooding rows

TABLE V
FEATURES AND LEAKAGE AUDIT

No.	Feature	Definition (v3)	Causal?	Test-day data?	Construction artifact	Max AUC (attack)	Max miss. AUC	Core	Decision
1	implied_speed_mps	haversine to previous fix / max(dt, 0.1 s)	yes	own past	ghost: missing (1-row track); flood: speed near 0	–	–	✓	keep
2	speed_consistency_delta	|implied speed - velocity|	yes	own past	ghost: missing	–	–	✓	keep
3	vrate_consistency_delta	|d(baroalt)/dt - vertrate|	yes	own past	ghost: missing; vel: vertrate filled, boost at least 5 m/s	–	–	✓	keep
4	inter_arrival_s	unfloored gap to previous message	yes	own past	ghost: missing; flood: sub-second gaps off the 10 s grid	–	–	✓	keep
5	turn_rate_deg_s	wrapped heading change / max(dt, 0.1 s)	yes	own past	ghost: missing	–	–	✓	keep
6	squawk_changed	previous and current code present and different	yes	own past	none (no attack edits squawk)	–	–		exclude
7	region_avg_inter_arrival	clean mean gap of the 10 deg cell over the chunk; median if unmapped	no	chunk	gps: re-mapped to spoofed cell (and client)	–	–	✓	keep
8	is_low_coverage_region	regional mean gap above chunk Q75	no	chunk	gps: via region lookup	–	–		exclude
9	icao24_valid_format	address is 6 lowercase hex digits	yes	no	none (constant; ghost ids valid)	–	–		exclude
10	track_message_count	running count of the address in the chunk	yes	own past	ghost: always 1	–	–		exclude
11	is_stale	time - lastcontact over 15 s	yes	no	none	–	–	✓	keep
12	is_physically_implausible	implied speed over 400 m/s	yes	own past	v2 only: benign rows over 400 m/s dropped	–	–		exclude
13	is_climbing_or_descending	|vertrate| over 2.5 m/s	yes	no	vel: boost of at least 5 m/s	–	–		exclude

Audit data: placeholder; audit numbers pending a run of leakage_audit.py on real data. Flag (bold): single-feature ROC AUC ≥ 0.95 or missing-indicator AUC ≥ 0.9 for some attack class vs benign; AUC reported as $\max(\mathrm{AUC}, 1-\mathrm{AUC})$ with missing values filled with -1. Causal: uses only the row and earlier messages of the same aircraft. Test-day data: own past = earlier messages of the same aircraft; chunk = statistic over all clean rows of the chunk, including later ones.

of that file, rows without the token were detected at 1.0%. The floor in (1) removes the token: after reprocessing, all four rate features are missing for –% of flooding rows and –% of benign rows in the test sample.

Equation (2) also exposed a labeling problem. The original message of each group is unmodified, so about $1/7$ of flooding rows could not be distinguished from benign traffic by any model, and flooding recall under the earlier labeling could not exceed approximately 0.857; a centralized LightGBM reached approximately that value. In the corrected pipeline only the appended duplicates are labeled `flooding`. Table XII compares models trained and tested on the earlier and the corrected pipeline.

V. MODELS AND FEDERATED TRAINING

A. Models

The detector is a one-dimensional CNN over the ordered feature vector, with layers `Conv1d(1,32,k=3,p=1)` and `Conv1d(32,64,k=3,p=1)`, ReLU activations, flattening and a linear output over six classes, without pooling, normalization or dropout. With the full feature set it has 11,334 parameters, or 45.3 kB as float32, which every client sends and receives in every round. Convolution over an ordered feature list is a weak inductive bias, since adjacent features have no natural locality; the architecture was chosen for its small size, and a multilayer perceptron of similar size is a natural alternative that we do not evaluate.

Three models are compared on identical rows and features (Fig. 1):

- **Federated CNN**: FedAvg over the region-cell clients.
- **Centralized CNN**: the same architecture trained on the union of the clients' training rows. Its difference from the federated CNN is the effect of federated training as configured here, which includes a different optimization schedule (below).
- **Centralized LightGBM** [39]: gradient-boosted trees on the same pooled rows, a strong tabular reference. A federated tree ensemble would require a different aggregation protocol and is outside the scope of this study.

B. Training

Each client splits its sampled rows 80/20 into training and validation rows. In each FedAvg round all clients start from the global weights $w^{(t)}$, train locally with a fresh Adam optimizer to obtain $w_c^{(t)}$, and the server sets

$$w^{(t+1)} = \sum_{c \in S_t} \frac{n_c}{\sum_{c' \in S_t} n_{c'}} w_c^{(t)}, \tag{3}$$

where n_c is client c's number of training rows and S_t the participating clients. The centralized CNN uses the same scaler, class weights, optimizer, learning rate and batch size, but a single Adam optimizer over the pooled rows and early stopping; the two models therefore differ in the number of gradient steps and in optimizer state, which Table IX reports. LightGBM uses balanced sample weights and early stopping on the validation rows. Table VI lists all settings, the selection rules and software versions.

Three steps use pooled information that a real federation would have to obtain differently: the feature scaler and the balanced class weights are computed on the full capped sample, including validation rows and the rows of cells that

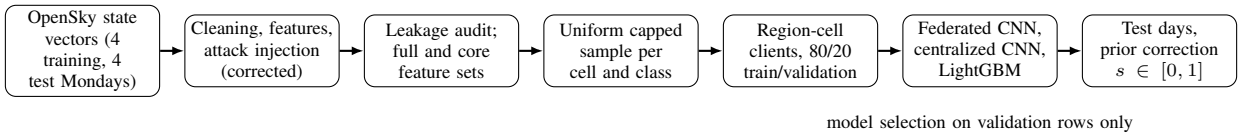


Fig. 1. Study workflow. Test days are used only for the reported scores; the class prior for correction comes from the training days.

TABLE VI
TRAINING CONFIGURATION

Setting	Value
Training sample	– rows from – raw rows; – sampler, cap – rows per (cell, class), budget –, seed 42
Train / validation	80/20 per client; – / – rows
Clients	– region cells with $\geq$– rows
Seeds	– (–): init, splits, shuffling, bagging
CNN	Conv1d 32, 64 ch., $k = 3$; – params, – bytes (–)
Optimizer	Adam, lr 10^{-3}, batch 512, balanced class weights
Scaler	Standardization fitted on the pooled training sample
FedAvg	– rounds, – local epochs, participation 1.0
Communication	– in total (up and down), – per round
Central CNN	$\leq$– epochs, early stop patience –
LightGBM	– leaves, lr –, $\leq$– trees, early stop – rounds, subsample 0.8, colsample 0.9
CNN selection	Round or epoch with the best unweighted validation accuracy of the uncorrected prediction ($s = 0$)
LightGBM selection	Early stopping on class-weighted multi_logloss of the validation rows
Training cost (full set)	Fed. CNN – steps, – min; Central CNN – steps, – min; LightGBM – iterations, – min
Prior correction	$s \in \{0, 0.1, \ldots, 1\}$, prior from all training-file rows
Selection data	Training-day validation rows only; test days never used
Software	Python –, PyTorch –, LightGBM –, scikit-learn –, pandas –, NumPy –

Training cost: mean over seeds; steps are optimizer updates summed over clients.

TABLE VII
CLIENT POPULATION

Quantity	Value
Clients (region cells)	–
Region cells dropped (<– rows)	–
Rows per client, min	–
Rows per client, median	–
Rows per client, max	–
Label JSD to global, mean	–
Label JSD to global, max	–
Clients missing $\geq$1 class (%)	–

Rows per client count training plus validation rows of the capped sample. JSD: Jensen-Shannon divergence (base 2, range 0 to 1) between a client's label distribution and the pooled one.



Fig. 2. Rows per client (training plus validation) in the capped sample, sorted in decreasing order.

are later dropped, and the cap itself is applied globally. Only raw rows are kept local; there is no secure aggregation or differential privacy. Communication counts one download and one upload of the model per client per round, $2RCB$ bytes for R rounds, C clients and B model bytes, which is – with the full feature set.

C. Client Heterogeneity

Region cells differ greatly in traffic volume and coverage. Table VII and Fig. 2 describe the client population, including the Jensen-Shannon divergence of each client's label distribution from the global one.

VI. Evaluation Protocol

A. Selection and Testing

All model selection uses training-day validation rows. The federated and centralized CNNs keep the round or epoch with the highest unweighted validation accuracy under the uncorrected decision rule ($s = 0$), and LightGBM stops on the class-weighted multiclass log loss. The test days are used only for the reported scores. Every configuration is run with – seeds, which vary model initialization, client train/validation splits, shuffling and tree bagging; the training sample and the test set are fixed across seeds, so the reported standard deviations reflect training randomness only and understate other sources of variability. For the federated-versus-centralized comparison, differences are computed per seed and summarized with a 95% t-interval.

B. Metrics

Metrics differ in whether they change when only the proportions of other classes change (Table VIII). Per-class recall, the false alarm rate (FAR) and the average attack recall do not, and are emphasized. Precision, macro F1, accuracy and the missed attack rate depend on the test mix, which is stated with them. Pooled metrics use all test rows; per-day metrics use each of the four test days.

C. Class-Prior Correction

Balanced class weights $w_k = N/(KN_k)$, computed on the capped sample, make the weighted share of every class in that sample equal to $1/K$, so the effective training prior is approximately uniform. Let $P_{\text{model}}(k \mid x)$ be the model's posterior and $\pi(k)$ a target class prior. Under label shift, the posterior under π is proportional to $P_{\text{model}}(k \mid x)\,\pi(k)/\pi_{\text{train}}(k)$ [37],

TABLE VIII
EVALUATION METRICS AND CLASS-MIX DEPENDENCE

Metric	Definition	Mix-dep.
Recall(k)	$TP(k)/(TP(k)+FN(k))$	No
Precision(k)	$TP(k)/(TP(k)+FP(k))$	Yes
Macro F1	Mean of per-class F1 over six classes	Yes
Accuracy	Correct predictions / all rows	Yes
FAR	Benign rows predicted as any attack / benign rows	No
Missed attack rate	Attack rows predicted benign / attack rows	Yes
Avg. attack recall	Mean recall over the five attack classes	No

which with a uniform effective training prior reduces to $P_{\text{model}}(k \mid x)\,\pi(k)$. We use

$$\hat{y} = \arg\max_{k}\ P_{\text{model}}(k \mid x)\,\pi(k)^{s}, \qquad s \in [0,1], \tag{4}$$

where $s = 0$ is the uncorrected model, $s = 1$ the full correction, and intermediate values interpolate in log-prior space. All strengths on the predefined grid $s \in \{0, 0.1, \ldots, 1\}$ are reported, so no strength is chosen on test data. The prior π is estimated from all rows of the four training days (–% benign) and never from test labels. The correction changes only the decision rule.

This π reflects the injection rate of the generator, not the frequency of real attacks, and the test days use the same rate, so on the test mix $s = 1$ is favored by construction. To examine a deployment in which attacks are much rarer, test rows are reweighted so that attacks make up 1% or 0.1% of traffic, keeping the five attack classes in their test proportions, while the correction still uses the training-day π; for comparison, an oracle correction uses the true deployment prior. The oracle maximizes expected accuracy, not macro F1, so at low prevalence it can suppress most alerts and score a lower macro F1 than the mismatched correction. FAR and per-class recall do not change under this reweighting; precision, macro F1 and the share of alerts that are real attacks do (Section VII-E).

VII. RESULTS

A. Federated and Centralized Training

Table IX reports the three models on the pooled test days at $s = 0$ and $s = 1$, with both feature sets, the per-seed federation gap, and the training cost of each model.

[PENDING RESULTS: state the measured federation gap with its 95% interval and whether it exceeds seed-to-seed variation; the remaining gap to LightGBM, attributed to model family; and the difference in gradient steps and training time.]

B. Feature Ablation and Pipeline Correction

Table X compares the full and core feature sets, Table XI gives LightGBM's feature importance, and Table XII compares the earlier and corrected pipelines.

[PENDING RESULTS: which classes lose recall without the excluded features; how flooding and ghost recall change between the pipelines; how much of the earlier performance came from the corrected artifacts.]

C. Per-Attack Results

Table XIII gives per-class precision, recall and F1 with test support, Table XIV the confusion matrix, and Table XV the two rule baselines.

[PENDING RESULTS: explain the hardest classes (subtle GPS offsets, velocity multipliers near 1.1, short replay delays) and how close the learned models are to the ghost and flooding rules.]

D. Generalization Across Days and Aircraft

Table XVI reports each test day separately. –% of test aircraft, carrying –% of test rows, also appear in the training days; Table XVII restricts the test set to the remaining aircraft.

[PENDING RESULTS: spread across days, whether the holiday days (23 and 30 December) differ, and the effect of restricting to unseen aircraft.]

E. Correction Strength and Attack Prevalence

Fig. 3 shows FAR, average attack recall and macro F1 over the strength grid for both feature sets, and Table XVIII shows macro F1 and the share of real alerts when attacks are rarer than in the test mix.

[PENDING RESULTS: shape of the trade-off; how far real-alert share falls at 1% and 0.1% prevalence; how much the oracle prior recovers.]

TABLE IX
FEDERATED VERSUS CENTRALIZED TRAINING ON IDENTICAL ROWS

Features	Model	s	Accuracy	FAR	Missed attacks	Attack recall	Macro F1
Full	Fed. CNN	0	–	–	–	–	–
		1	–	–	–	–	–
	Central CNN	0	–	–	–	–	–
		1	–	–	–	–	–
	LightGBM	0	–	–	–	–	–
		1	–	–	–	–	–
	Fed.−central	0		– –		– –	– –
	Fed.−central	1		– –		– –	– –
Core	Fed. CNN	0	–	–	–	–	–
		1	–	–	–	–	–
	Central CNN	0	–	–	–	–	–
		1	–	–	–	–	–
	LightGBM	0	–	–	–	–	–
		1	–	–	–	–	–
	Fed.−central	0		– –		– –	– –
	Fed.−central	1		– –		– –	– –

Mean ± standard deviation over – seeds, in %. Fed. CNN and Central CNN share architecture, scaler, class weights, optimizer, training rows and validation rows; only the training procedure differs. Fed.−central: paired difference per seed in percentage points, mean and 95% t-interval over seeds. Missed attacks: attack rows predicted benign over attack rows.

TABLE X
FULL VERSUS CORE FEATURE SET

						Recall					
s	Model	Features	Macro F1	FAR	Atk. rec.	Benign	Flooding	Ghost aircraft	GPS spoof	Replay	Velocity spoof
1	Fed. CNN	Full	–	–	–	–	–	–	–	–	–
		Core	–	–	–	–	–	–	–	–	–
	Central CNN	Full	–	–	–	–	–	–	–	–	–
		Core	–	–	–	–	–	–	–	–	–
	LightGBM	Full	–	–	–	–	–	–	–	–	–
		Core	–	–	–	–	–	–	–	–	–
0.5	Fed. CNN	Full	–	–	–	–	–	–	–	–	–
		Core	–	–	–	–	–	–	–	–	–
	Central CNN	Full	–	–	–	–	–	–	–	–	–
		Core	–	–	–	–	–	–	–	–	–
	LightGBM	Full	–	–	–	–	–	–	–	–	–
		Core	–	–	–	–	–	–	–	–	–

Mean ± standard deviation over – seeds, in %. The core set keeps implied speed, speed and vertical-rate consistency, inter-arrival time, turn rate, regional inter-arrival baseline and the stale flag.

TABLE XI
LIGHTGBM FEATURE IMPORTANCE (GAIN SHARE)

Feature	Full set	Core set
implied_speed_mps	–	–
speed_consistency_delta	–	–
vrate_consistency_delta	–	–
inter_arrival_s	–	–
turn_rate_deg_s	–	–
track_message_count	–	n/a
is_physically_implausible	–	n/a
squawk_changed	–	n/a
icao24_valid_format	–	n/a
is_stale	–	–
region_avg_inter_arrival	–	–
is_low_coverage_region	–	n/a
is_climbing_or_descending	–	n/a

Mean ± standard deviation over – seeds, in %. Share of total split gain; n/a: feature not in the core set. Correlated features can substitute for each other, so a low share does not rule out leakage.

TABLE XII
OLD (V2) VERSUS CORRECTED (V3) PIPELINE (FULL FEATURE SET)

Model	s	Data	Macro F1	FAR	Atk. rec.	Flooding rec.	Ghost rec.
LightGBM	0	v2	–	–	–	–	–
		v3	–	–	–	–	–
	1	v2	–	–	–	–	–
		v3	–	–	–	–	–
Fed. CNN	0	v2	–	–	–	–	–
		v3	–	–	–	–	–
	1	v2	–	–	–	–	–
		v3	–	–	–	–	–

Mean ± standard deviation over seeds, in %. v2: the earlier labelled files and test sample; v3: the corrected pipeline. Each version is evaluated on its own test sample, drawn the same way.

TABLE XIII
PER-CLASS PRECISION, RECALL AND F1 AT FULL CORRECTION ($s = 1$, FULL FEATURE SET)

		Fed. CNN			Central CNN			LightGBM		
Class	Support	P	R	F1	P	R	F1	P	R	F1
Benign		–	–	–	–	–	–	–	–	–
Flooding		–	–	–	–	–	–	–	–	–
Ghost aircraft		–	–	–	–	–	–	–	–	–
GPS spoof		–	–	–	–	–	–	–	–	–
Replay		–	–	–	–	–	–	–	–	–
Velocity spoof		–	–	–	–	–	–	–	–	–

Mean ± standard deviation over – seeds, in %. Support: held-out rows of the class.

TABLE XIV
CONFUSION MATRICES AT FULL CORRECTION ($s = 1$, FULL FEATURE SET)

	Fed. CNN (predicted)						Central CNN (predicted)					
Actual	Ben.	Flood	Ghost	GPS	Repl.	Vel.	Ben.	Flood	Ghost	GPS	Repl.	Vel.
Benign	–	–	–	–	–	–	–	–	–	–	–	–
Flooding	–	–	–	–	–	–	–	–	–	–	–	–
Ghost aircraft	–	–	–	–	–	–	–	–	–	–	–	–
GPS spoof	–	–	–	–	–	–	–	–	–	–	–	–
Replay	–	–	–	–	–	–	–	–	–	–	–	–
Velocity spoof	–	–	–	–	–	–	–	–	–	–	–	–

Row-normalised (each row sums to 100%), mean over – seeds.

TABLE XV
SINGLE-RULE BASELINES ON THE TEST SET

Class	Rule	Recall	Prec.	FAR
Ghost aircraft	All four rate features missing	–	–	–
Flooding	Inter-arrival time below 1 s	–	–	–

In %, no training. Recall and precision for the rule's class; FAR: benign rows the rule fires on over benign rows. Rate features: implied speed, speed and vertical-rate consistency, turn rate. Missing inter-arrival times do not fire the flooding rule.

TABLE XVI
RESULTS PER HELD-OUT DAY AT FULL CORRECTION ($s = 1$, FULL FEATURE SET)

		Fed. CNN			Central CNN			LightGBM		
Day	Rows	Macro F1	FAR	Atk. rec.	Macro F1	FAR	Atk. rec.	Macro F1	FAR	Atk. rec.
–	–	–	–	–	–	–	–	–	–	–
–	–	–	–	–	–	–	–	–	–	–
–	–	–	–	–	–	–	–	–	–	–
–	–	–	–	–	–	–	–	–	–	–

Mean ± standard deviation over – seeds, in %. The day of a row is the UTC date of its last position update (the replay attack shifts the time field but not this one).

TABLE XVII
TEST ROWS FROM AIRCRAFT NEVER SEEN IN TRAINING ($s = 1$, FULL FEATURE SET)

	All test rows			Unseen aircraft		
Model	Macro F1	FAR	Atk. rec.	Macro F1	FAR	Atk. rec.
Fed. CNN	–	–	–	–	–	–
Central CNN	–	–	–	–	–	–
LightGBM	–	–	–	–	–	–

Mean ± standard deviation over – seeds, in %. –% of test rows and –% of test aircraft addresses also occur in the training files. Unseen-aircraft subset: – rows (benign –, flooding –, ghost aircraft –, gps spoof –, replay –, velocity spoof –). Ghost rows carry fabricated addresses and are therefore almost all in this subset.

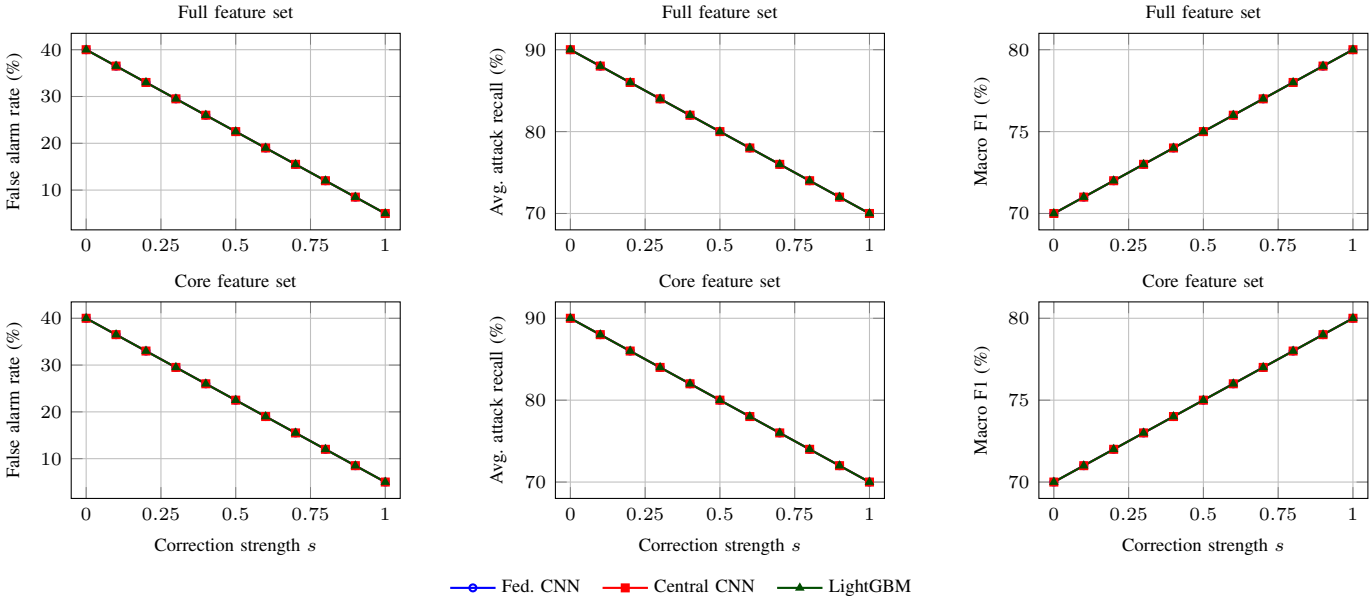


Fig. 3. False alarm rate, average attack recall and macro F1 against the class-prior correction strength s (pooled held-out days). Top: full feature set; bottom: core feature set. Error bars: standard deviation over seeds.

TABLE XVIII
RESULTS UNDER ASSUMED DEPLOYMENT ATTACK PREVALENCE (FULL FEATURE SET)

		Test mix		$\rho = 1\%$		$\rho = 0.1\%$	
Model	s	Macro F1	Real alerts	Macro F1	Real alerts	Macro F1	Real alerts
Fed. CNN	0	–	–	–	–	–	–
	0.5	–	–	–	–	–	–
	1	–	–	–	–	–	–
	oracle	–	–	–	–	–	–
Central CNN	0	–	–	–	–	–	–
	0.5	–	–	–	–	–	–
	1	–	–	–	–	–	–
	oracle	–	–	–	–	–	–
LightGBM	0	–	–	–	–	–	–
	0.5	–	–	–	–	–	–
	1	–	–	–	–	–	–
	oracle	–	–	–	–	–	–

Mean $\pm$ standard deviation over – seeds, in %. Test rows are reweighted (not resampled) so that attacks make up a share ρ of traffic, the five attack classes keeping their test proportions; models are not retrained. Rows s: correction with the training-day prior, as a deployment would apply it. Oracle: correction at $s = 1$ with the true deployment prior. Real alerts: share of rows flagged as any attack that are attacks. Recall and false alarm rate do not change under reweighting.

VIII. Discussion

[PENDING RESULTS: five short paragraphs from the results: (1) how much of the earlier performance came from artifacts (ablation, pipeline comparison, rule baselines); (2) the measured cost of federated training as configured; (3) why LightGBM performs as it does on tabular features with missing-value structure; (4) the false-alarm versus recall trade-off and what it implies at realistic prevalence; (5) comparison with the studies in Table I, noting differences in data, attacks and evaluation. Keep findings separate from explanations.]

IX. Limitations

Synthetic attacks. Benign traffic is real, but every attack is synthetic, generated per row by the rules in Table IV. The results measure detection of those rules; they do not establish detection rates against real or adaptive adversaries, who could shape messages to stay within normal feature ranges. Attacks are isolated perturbations, not coherent aircraft-level episodes, and the attack prevalence in training and testing is set by the generator.

Temporal and aircraft generalization. Training and test days are disjoint and ordered in time, but the test days all fall in December, and the same aircraft, routes and receivers appear in both; only part of the test set comes from aircraft unseen in training. The same generator is used for training and testing, so the test measures a later period under the same threat model, not an independently designed one.

Uncertainty. Variability is estimated over – training seeds with a fixed training sample and a fixed test sample, so it does not include sampling variability of the data or of the attack injection.

Federation realism. Clients are 10° grid cells, not real organizations, whose coverage would overlap and not follow a grid. Federation is simulated in one process with full participation and no dropouts, stragglers or network faults, and the scaler, class weights and sampling cap use pooled information. Only raw-record locality is claimed: there is no secure aggregation or differential privacy, and attacks on the model updates are not tested.

Features. The regional baseline uses data from the whole chunk, and filling missing values with -1 conflates different causes of missingness and is in range for turn rate. The core set removes the known artifacts, but low single-feature separability does not rule out leakage, because combinations of features can still encode the generator.

Scope. Latency, throughput and real-time operation were not measured. No claim of operational readiness is made; the intended use is situational awareness at receivers, not a replacement for certified surveillance or collision-avoidance systems.

X. Conclusion and Future Work

[PENDING RESULTS: conclusion from the final results: what survives the leakage audit, the measured cost of federated training, the trade-off offered by prior correction at realistic prevalence, and the classes that remain difficult.]

Future work includes coherent episode-level attacks generated per aircraft, multi-message ghost tracks and flooding generated at the raw message rate, a trailing-window regional baseline, explicit missingness indicators, aircraft-disjoint and region-disjoint splits, federated preprocessing (federated scaling and label-count aggregation), algorithms for non-IID clients such as FedProx [26], secure aggregation and differential privacy with their accuracy cost, and validation against independently designed or real attack traces.

Data and Code Availability

The OpenSky Network historical state-vector data are available from the OpenSky Network [8]. Code for preprocessing, attack injection, the leakage audit, training and evaluation, with the exact commands, configuration, random seeds and software versions, is available at [PENDING RESULTS: repository URL].

References

[1] M. Strohmeier, M. Schäfer, V. Lenders, and I. Martinovic, "Realities and challenges of NextGen air traffic management: The case of ADS-B," *IEEE Communications Magazine*, vol. 52, no. 5, pp. 111–118, 2014.

[2] U.S. Federal Aviation Administration, "14 CFR section 91.225: Automatic dependent surveillance-broadcast (ADS-B) out equipment and use," Code of Federal Regulations, Title 14, 2010, final rule published 2010; equipage required from 1 January 2020. Accessed 27 October 2026. [Online]. Available: https://www.law.cornell.edu/cfr/text/14/91.225

[3] A. Costin and A. Francillon, "Ghost in the air(traffic): On insecurity of ADS-B protocol and practical attacks on ADS-B devices," in *Black Hat USA*, Las Vegas, NV, USA, Jul. 2012. [Online]. Available: https://www.eurecom.fr/publication/3788

[4] M. Schäfer, V. Lenders, and I. Martinovic, "Experimental analysis of attacks on next generation air traffic communication," in *Applied Cryptography and Network Security (ACNS 2013)*, ser. Lecture Notes in Computer Science, vol. 7954. Springer, 2013, pp. 253–271.

[5] M. Strohmeier, V. Lenders, and I. Martinovic, "On the security of the automatic dependent surveillance-broadcast protocol," *IEEE Communications Surveys & Tutorials*, vol. 17, no. 2, pp. 1066–1087, 2015.

[6] W. Ahmed, A. Masood, and J. Manzoor, "Towards secure air traffic surveillance: A survey on ADS-B threats, existing solutions, and future research," *IEEE Open Journal of the Computer Society*, 2026.

[7] M. S. Ngamboe Mvogo, X. Niu, B. Joly, S. P. Biegler, P. Berthier, R. Benito, G. Rice, J. M. Fernandez, and G. Nicolescu, "CABBA: Compatible authenticated bandwidth-efficient broadcast protocol for ADS-B," *International Journal of Critical Infrastructure Protection*, vol. 48, p. 100728, 2025.

[8] M. Schäfer, M. Strohmeier, V. Lenders, I. Martinovic, and M. Wilhelm, "Bringing up OpenSky: A large-scale ADS-B sensor network for research," in *Proceedings of the 13th International Symposium on Information Processing in Sensor Networks (IPSN '14)*. Berlin, Germany: IEEE Press, 2014, pp. 83–94.

[9] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. Aguera y Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics (AISTATS)*, ser. Proceedings of Machine Learning Research, A. Singh and J. Zhu, Eds., vol. 54. PMLR, 2017, pp. 1273–1282. [Online]. Available: https://proceedings.mlr.press/v54/mcmahan17a.html

[10] T. Li, B. Wang, F. Shang, J. Tian, and K. Cao, "Threat model and construction strategy on ADS-B attack data," *IET Information Security*, vol. 14, no. 5, pp. 542–552, 2020.

[11] J. Wang, Y. Zou, and J. Ding, "ADS-B spoofing attack detection method based on LSTM," *EURASIP Journal on Wireless Communications and Networking*, vol. 2020, p. 160, 2020.

[12] W. Ahmed, A. Masood, J. Manzoor, and S. Akleylek, "Automatic dependent surveillance-broadcast (ADS-B) anomalous messages and attack type detection: Deep learning-based architecture," *PeerJ Computer Science*, vol. 11, p. e2886, 2025.

[13] D. Arp, E. Quiring, F. Pendlebury, A. Warnecke, F. Pierazzi, C. Wressnegger, L. Cavallaro, and K. Rieck, "Dos and don'ts of machine learning in computer security," in *31st USENIX Security Symposium (USENIX Security 22)*. Boston, MA: USENIX Association, 2022, pp. 3971–3988.

[14] S. Kapoor and A. Narayanan, "Leakage and the reproducibility crisis in machine-learning-based science," *Patterns*, vol. 4, no. 9, p. 100804, 2023.

[15] E. Habler and A. Shabtai, "Using LSTM encoder-decoder algorithm for detecting anomalous ADS-B messages," *Computers & Security*, vol. 78, pp. 155–173, 2018.

[16] A. Fried and M. Last, "Facing airborne attacks on ADS-B data with autoencoders," *Computers & Security*, vol. 109, p. 102405, 2021.

[17] H. Ould Slimane, S. Benouadah, K. Al Shamaileh, V. K. Devabhaktuni, and N. Kaabouch, "ADS-B message injection attack on UAVs: Assessment of SVM-based detection techniques," in *2022 IEEE International Conference on Electro Information Technology (eIT)*, 2022.

[18] T. Talaei Khoei, H. Ould Slimane, K. Al Shamaileh, V. K. Devabhaktuni, and N. Kaabouch, "Detecting injection attacks in ADS-B devices using RNN-based models," in *2024 Integrated Communications, Navigation and Surveillance Conference (ICNS)*, 2024.

[19] N. Çevik and S. Akleylek, "Anomaly detection system for ADS-B data: Attack vectors and machine learning models," *Internet of Things*, vol. 29, p. 101416, 2025.

[20] M. Ngamboé, J.-S. Marrocco, J.-Y. Ouattara, J. M. Fernandez, and G. Nicolescu, "New machine learning approaches for intrusion detection in ADS-B," in *44th Digital Avionics Systems Conference (DASC 2025)*, Montreal, QC, Canada, Sep. 2025, also arXiv:2510.08333.

[21] X. Ying, J. Mazer, G. Bernieri, M. Conti, L. Bushnell, and R. Poovendran, "Detecting ADS-B spoofing attacks using deep neural networks," in *2019 IEEE Conference on Communications and Network Security (CNS)*, 2019, pp. 187–195.

[22] T. Li and B. Wang, "Sequential collaborative detection strategy on ADS-B data attack," *International Journal of Critical Infrastructure Protection*, vol. 24, pp. 78–99, 2019.

[23] S. Khan, A. Braeken, P. Kumar, and A. Gurtov, "POSTER: FL-Guard: A federated learning based ground-air secure communication model for future aviation network," Poster, Network and Distributed System Security Symposium (NDSS), 2022. [Online]. Available: https://www.ndss-symposium.org/wp-content/uploads/NDSS2022Poster_paper_26.pdf

[24] Q. Yang, Y. Liu, T. Chen, and Y. Tong, "Federated machine learning: Concept and applications," *ACM Transactions on Intelligent Systems and Technology*, vol. 10, no. 2, pp. 1–19, 2019.

[25] Y. Zhao, M. Li, L. Lai, N. Suda, D. Civin, and V. Chandra, "Federated learning with non-IID data," 2018.

[26] T. Li, A. K. Sahu, M. Zaheer, M. Sanjabi, A. Talwalkar, and V. Smith, "Federated optimization in heterogeneous networks," in *Proceedings of Machine Learning and Systems (MLSys)*, 2020, pp. 429–450.

[27] L. Zhu, Z. Liu, and S. Han, "Deep leakage from gradients," in *Advances in Neural Information Processing Systems (NeurIPS 2019)*, 2019, pp. 14 774–14 784.

[28] K. Bonawitz, V. Ivanov, B. Kreuter, A. Marcedone, H. B. McMahan, S. Patel, D. Ramage, A. Segal, and K. Seth, "Practical secure aggregation for privacy-preserving machine learning," in *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. ACM, 2017, pp. 1175–1191.

[29] M. Abadi, A. Chu, I. Goodfellow, H. B. McMahan, I. Mironov, K. Talwar, and L. Zhang, "Deep learning with differential privacy," in *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. ACM, 2016, pp. 308–318.

[30] T. D. Nguyen, S. Marchal, M. Miettinen, H. Fereidooni, N. Asokan, and A.-R. Sadeghi, "DÏoT: A federated self-learning anomaly detection system for IoT," in *39th IEEE International Conference on Distributed Computing Systems (ICDCS)*, 2019.

[31] P. Ruzafa-Alcazar, P. Fernandez-Saura, E. Marmol-Campos, A. Gonzalez-Vidal, J. L. Hernandez-Ramos, J. Bernal-Bernabe, and A. F. Skarmeta, "Intrusion detection based on privacy-preserving federated learning for the industrial IoT," *IEEE Transactions on Industrial Informatics*, vol. 19, no. 2, pp. 1145–1154, 2023.

[32] M. Abdel-Basset, N. Moustafa, H. Hawash, I. Razzak, K. M. Sallam, and O. M. Elkomy, "Federated intrusion detection in blockchain-based smart transportation systems," *IEEE Transactions on Intelligent Transportation Systems*, vol. 23, no. 3, pp. 2523–2537, 2022.

[33] M. Kesici, B. Pal, and G. Yang, "Detection of false data injection attacks in distribution networks: A vertical federated learning approach," *IEEE Transactions on Smart Grid*, vol. 15, no. 6, pp. 5952–5964, 2024.

[34] A. Khraisat, A. Alazab, S. Singh, T. Jan, and A. Gomez, "Survey on federated learning for intrusion detection system: Concept, architectures, aggregation strategies, challenges, and future directions," *ACM Computing Surveys*, vol. 57, no. 1, p. 7, 2024.

[35] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic minority over-sampling technique," *Journal of Artificial Intelligence Research*, vol. 16, pp. 321–357, 2002.

[36] I. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville, and Y. Bengio, "Generative adversarial nets," in *Advances in Neural Information Processing Systems (NIPS 2014)*, 2014, pp. 2672–2680.

[37] M. Saerens, P. Latinne, and C. Decaestecker, "Adjusting the outputs of a classifier to new a priori probabilities: A simple procedure," *Neural Computation*, vol. 14, pp. 21–41, 2002.

[38] C. Elkan, "The foundations of cost-sensitive learning," in *Proceedings of the International Joint Conference on Artificial Intelligence (IJCAI)*, 2001, pp. 973–978.

[39] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu, "LightGBM: A highly efficient gradient boosting decision tree," in *Advances in Neural Information Processing Systems (NIPS 2017)*, 2017, pp. 3146–3154.